

REPRODUCIBLE LLM-GUIDED DISCOVERY OF ENTANGLEMENT-ASSISTED QUANTUM CODES

Anonymous authors

Paper under double-blind review

ABSTRACT

LLM-guided program evolution can produce mathematical constructions, but verifying a result and reproducing the search that produced it are separate requirements. We study both for entanglement-assisted quantum error-correcting codes (EAQECCs), using targets derived from tabulated EAQECC bounds and an exact symplectic evaluator. The core construction archive contains 115 verified codes, a closed-form construction $[[n, 1, n-1; n-q-1]]_q$ for every prime power at the stated admissible lengths, and ten nonexistence results backed by DRAT certificates. The construction and ebit lifting settle 54 listed gaps; applying the known EA-Plotkin bound tightens 398 listed upper bounds. An auditor checks the archived computational results and replays fixed search programs using an evaluation-based stopping rule. It does not reproduce the LLM sampling that generated those programs. The pipeline supports researcher proposals that change the search representation and can be developed through program-level feedback. Under identical guidance to search the symplectic dual, a four-block comparison increases held-out success from 50.0% to 68.8% at $n = 9$ and from 15.6% to 50.0% at $n = 11$ when independent proposals are replaced by iterative program development. One validation-selected later-generation program achieves 75.0% success at each of $n = 13, 15$.

1 INTRODUCTION

LLM-guided program search has produced new mathematical constructions and algorithms (Romera-Paredes et al., 2024; Novikov et al., 2025). Assessing such results requires answering three questions: whether the proposed object is valid, whether it improves on prior constructions, and what the model contributed beyond the initial program and task specification. These questions require different evidence. A verified output does not by itself establish novelty, and rerunning a final program does not reproduce the evolutionary process that generated it.

We study these distinctions using entanglement-assisted quantum error-correcting codes (EAQECCs). Their parameters can be computed exactly from a finite list of generators, while tabulated EAQECC bounds provide a specific reference for measuring improvements. Figure 1 summarizes our researcher-guided pipeline for program evolution, exact verification and nonexistence checking. The artifact records the mathematical results and the evidence needed to check each type of claim.

Several improvements concern constructions published in 2020 and 2022 that remained listed in the EAQECC tables accessed on 17 July 2026 (Grassl et al., 2020; Luo et al., 2022). Our constructions and certified exclusions close nine previously tabulated qubit distance gaps (Table 1). The prior constructions predate our table access by approximately four to six years. Their dates document available constructions rather than a complete history of improvement attempts.

Contributions.

- **A search problem with explicit verification criteria.** We construct targets from tabulated EAQECC bounds. The archived qubit table contains 563 entries whose distance bounds differ by one. We distinguish closing a listed gap, improving a lower bound and constructing a code at a parameter cell absent from the table (Section 2).

- **A researcher-guided pipeline and reproduction protocol.** Researchers can revise construction ideas and campaign targets through archived task specifications. The auditor checks codes, family instances, table comparisons, nonexistence evidence and prompt provenance. A virtual clock tied to evaluator calls makes fixed-program search replayable under an evaluation-based stopping rule; negative controls test the auditor’s response to damaged inputs (Section 3.5).
- **EAQECC constructions and nonexistence results.** A closed form at distance $n - 1$ settles 54 listed gaps, including qutrit entries obtained by ebit lifting. We also archive finite codes using less entanglement and nonexistence results that establish minimum entanglement at specified lengths. Applying the known EA-Plotkin bound tightens 398 listed upper bounds (Tables 1 and 2).
- **Research-guided program development with measured outcomes.** Two generation policies receive the same initial program and researcher proposal to search the symplectic dual. Iterative development achieves higher mean held-out success than independent proposals, with the larger difference at length 11. A selected later-generation program also generalizes to lengths 13 and 15 (Section 5).

2 BACKGROUND AND RELATED WORK

2.1 ENTANGLEMENT-ASSISTED QUANTUM ERROR-CORRECTING CODES

Quantum error correction protects quantum information sent through noisy communication links. Early stabilizer constructions imposed algebraic compatibility constraints that limited which classical codes could be reused. Brun et al. (2006) showed that entanglement shared in advance between sender and receiver relaxes these constraints, enabling a broader class of quantum codes. This is the motivation for entanglement-assisted quantum error-correcting codes (EAQECCs).

A code with parameters $[[n, k, d; c]]_q$ encodes k logical qudits into n transmitted q -level systems using c shared maximally entangled pairs. For qubits ($q = 2$), each pair supplies one ebit. The receiver’s halves are assumed noiseless and are not counted in n .

The distance d determines the code’s guaranteed error protection. With ideal decoding, it corrects arbitrary errors on up to $\lfloor (d - 1)/2 \rfloor$ transmitted qudits, or up to $d - 1$ *erasures*, where the lost qudits’ locations are known (Brun et al., 2006; Grassl et al., 2022). Increasing d at fixed (q, n, k, c) strengthens protection without increasing the transmission or entanglement budget. For example, our $[[12, 1, 11; 8]]_2$ code raises the listed distance from 9 to 11, increasing the guaranteed correction of arbitrary errors from four to five transmitted qubits with the same resources (Table 1). A verified code thus provides an encoding specification for a quantum link with given resource and error-protection requirements. Reducing c at fixed (n, k, d) instead lowers the entanglement needed before transmission (Luo et al., 2022).

2.2 CODE DISCOVERY AS PROGRAM SEARCH

We search linear stabilizer EAQECCs represented by generator matrices over $\mathbb{F}_q$. The LLM revises a search program that proposes matrices; an exact evaluator computes their (k, c, d) . At fixed (q, n, k, c) , reference tables give a constructed lower distance bound d_ℓ and an upper bound d_u (Grassl, 2026; Luo et al., 2022). Finding $d > d_\ell$ improves the lower bound, while reaching d_u closes the gap. We also construct codes at unlisted cells and minimize entanglement, denoted $c_{\min}^s(n, k, d)$ within our stabilizer class. Applicable bounds filter impossible targets (Grassl et al., 2022); Appendix A gives the representation, notation and comparison rules.

FunSearch evolves programs that construct mathematical objects or make algorithmic decisions, using an evaluator to select useful revisions. It produced larger cap-set constructions in extremal combinatorics and improved heuristics for online bin packing (Romera-Paredes et al., 2024). Its output is an executable construction procedure, allowing researchers to inspect a solution strategy and develop the mathematical ideas it uses.

AlphaEvolve extends evaluator-guided program evolution to broader scientific and computational tasks (Novikov et al., 2025; Georgiev et al., 2025). Its applications include searches for matrix-multiplication algorithms, data-center scheduling, hardware-circuit simplification and optimization

of model-training kernels. LLMs edit executable programs, and measured performance guides subsequent proposals. We use AlphaEvolve as the evolution engine for EAQECC discovery, supplying the target parameters, exact evaluator and researcher-directed construction ideas.

Quantum-code search has also used genetic and reinforcement-learning methods. Webster & Browne (2024) mutate and recombine binary representations of stabilizer codes to optimize distance and performance under biased noise. Reinforcement learning has been used to discover quantum codes, optimize tensor-network constructions, and jointly design codes and encoders (Poulsen Nautrup et al., 2019; Su et al., 2025; Mauron et al., 2023; Olle et al., 2024). Our search instead evolves the executable procedure that proposes codes. This permits changes to construction rules and search policies while retaining the same exact verification interface; the controlled study measures the contribution of iterative program feedback.

3 THE DISCOVERY PIPELINE

Our discovery pipeline combines human-in-the-loop (HITL) research with LLM-guided program evolution and exact verification. Researchers propose construction ideas, inspect verified instances and revise the task between campaigns. Figure 1 connects these interventions to the automated search and evaluation loop. Constructed codes establish lower distance bounds; SAT refutations with checked certificates lower upper bounds. Independent verification and archived evidence support both routes. The resulting constructions can inform further searches and human mathematical generalization.

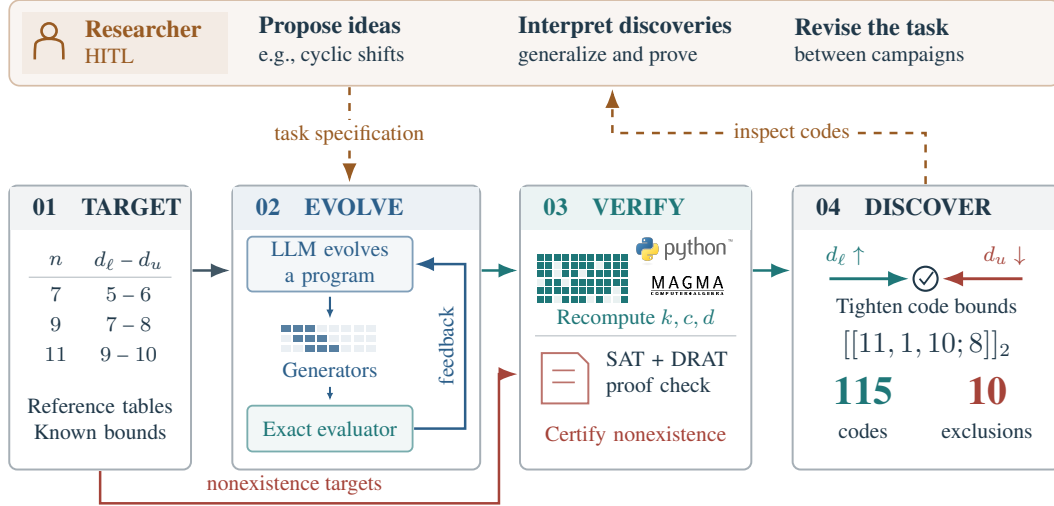


Figure 1: Human-in-the-loop discovery and verification.

3.1 TARGET SELECTION

We compare our constructions with the bounds listed in the EAQECC tables (`codetables.de`), accessed on 17 July 2026. These reference bounds define listed-gap and absent-cell targets, filtered using the EA-Plotkin and applicable Singleton bounds. EA-Plotkin also tightens 398 upper bounds in the reference qubit table. Appendix A.3 gives the table records and filters. The artifact preserves these comparison data and uses them by default. A second copy, retrieved on 10 September 2026, contains identical intervals at all 11,640 qubit and qutrit entries.

3.2 LLM-GUIDED PROGRAM EVOLUTION

The discovery campaigns use AlphaEvolve (Novikov et al., 2025) through Google Cloud Vertex AI. We provide a seed program, a task specification and an exact evaluator; AlphaEvolve manages model-generated revisions and uses returned scores and feedback to guide further program evolu-

tion. The controller evaluates candidates in parallel and archives their source and results. An LLM modifies a designated block of the seed. Each candidate program searches the target list and returns up to 16 proposals. The external evaluator checks these proposals, assigns the candidate its best proposal score, and supplies numerical and textual feedback for subsequent program mutations. The score rewards newly achieved targets and penalizes mismatched (k, c) parameters. For a valid signature, progress is measured by the number of nontrivial logical operators below the target distance. This provides a finer search signal than the minimum distance, which often stays constant across mutations.

Each candidate runs in its own subprocess with a fixed 240 s budget and RNG seed. Failures receive a low score, and the campaign continues. The same interface supports fixed-program replay (Section 3.5). The four campaign logs record 401 completed candidate searches, corresponding to 26.7 h of allocated search budget (Appendix B.1).

Researchers can add construction ideas through the task specification while retaining the evaluator and execution interface. In this study, the initial program used annealing, while the researcher-written specification suggested cyclic shifts among several directions. Evolution implemented this idea and produced finite code instances. Researchers then derived a general family and incorporated it into later task specifications, which requested further constructions. A write-back step also reused the best program as the next seed. The archived specifications document these updates (Appendix B.3). This permits a research result to guide the next campaign; those later seeds are consequently not independent baselines for rediscovering the earlier instances.

3.3 INDEPENDENT VERIFICATION

The Magma export represents each code as a basis of the symplectic dual $L = S^\perp$, where S spans its stabilizer generators (Appendix A.1), without derived quantities. Magma recomputes the symplectic dual, radical, and k, c, d independently of the Python implementation. The full export contains the 115 core codes at $q = 2, 3, 4, 5$ and 33 family instances. All 148 objects were re-verified with 0 mismatches under Magma V2.28-20. A separate binary linear-algebra implementation also checks the two solver-refined qubit results below. Magma provides additional corroboration; every archived code can also be checked in Python (Appendix C).

3.4 NONEXISTENCE CHECKING

To test whether an $[[n, k, d; c]]_q$ code exists, we encode the lower-dimensional subspace $L = S^\perp$ in symplectic normal form, with constraints that force the target signature by construction rather than by a subsequent filter, so that every satisfying assignment is already an EAQECC. The distance constraints account for most of the encoding, and the encoding is given in Appendix B.2.

For an unsatisfiable encoding, the SAT solver produces a DRAT proof, a certificate of unsatisfiability checked independently by DRAT-trim (Wetzler et al., 2014). Together with the encoding’s completeness, this establishes the corresponding code nonexistence result. The qutrit encoding uses a coordinate-normalization lemma to preserve completeness. Appendix B.2 gives the proof scope and encoding, and Section 4.3 reports the certified results.

3.5 REPRODUCIBILITY

The auditor checks codes, family instances, nonexistence evidence, reference bounds and prompt provenance, and reports which checks were completed (Appendix C). Fixed-program replay replaces wall time with a virtual clock tied to evaluator calls, preserving each program’s loop conditions and stopping point for a given seed and nominal budget. Three archived programs with four seeds each reproduce all twelve stored proposal lists; one- and four-worker executions also agree. This replays recorded program executions, not the LLM sampling that generated them. Appendix D gives the controlled-study protocol and execution costs.

4 RESULTS

Algorithm 1 presents one of the highest-scoring archived programs from the AlphaEvolve construction campaign. Figure 2 compares three changes with its campaign seed: progress-dependent target selection, reuse of a stored proposal for initialization, and cyclic rotation of a whole generator during repair. The seed already stored best proposals for output; the evolved program also uses those records to guide subsequent search. Shading marks these principal changes. The controlled policy comparison in Section 5 uses separately generated programs, including Algorithm 2 in Appendix D.4.

Algorithm 1 Search program evolved by AlphaEvolve.

Require: targets $\mathcal{T}$, time allowance B
 $\Phi(r, t) = 5000|r.c - c_t| + 1000|r.k - k_t| + r.\text{offending}$
1: $best \leftarrow \{\}; b_t \leftarrow 0$ for all $t \in \mathcal{T}$
2: **while** time remains **do**
3: $\pi_t \propto n_t^{-1.5}(1 + 10b_t)$; sample $t \sim \pi$ ▷ Use recorded progress to change target selection.
4: $n \leftarrow n_t$; $s \leftarrow n_t - k_t + c_t$; $u \sim \text{Uniform}[0, 1)$
5: **if** $u < 0.2$, $t \in best$ and $best[t].o > 0$ **then** ▷ Resume from this target’s stored best proposal.
6: $G \leftarrow \text{copy}(best[t].G)$
7: **else** ▷ Expand construction choices.
8: $G \leftarrow \text{CONSTRUCT}(t, u)$
9: **end if**
10: **if** G is undefined or rank $G \neq s$ **then**
11: $G \leftarrow \text{RANDOMSTABILIZER}(n, s)$
12: **end if**
13: $o \leftarrow \Phi(\text{EVALUATE}(n, G, d_t), t)$; $\text{KEEPBEST}(best, t, G, o)$
14: **if** $o = 0$ **then continue**; $\tau \leftarrow 30$; $f \leftarrow 0$
15: **while** $f < 1500$ and time remains **do**
16: $\tau \leftarrow \max(0.1, 0.995\tau)$
17: $G' \leftarrow \text{MIXEDMOVE}(G)$ ▷ Add cyclic rotation of a whole generator row.
18: **if** rank $G' \neq s$ **then** $f \leftarrow f + 1$; **continue**
19: $o' \leftarrow \Phi(\text{EVALUATE}(n, G', d_t), t)$; $\Delta \leftarrow o' - o$
20: Set $(G, o) \leftarrow (G', o')$ with probability $\min\{1, \exp(-\Delta/\tau)\}$
21: $f \leftarrow 0$ on an accepted strict improvement, otherwise $f \leftarrow f + 1$
22: $\text{KEEPBEST}(best, t, G, o)$; **if** $o = 0$ **then break**
23: **end while**
24: $b_t \leftarrow \text{PROGRESSBONUS}(best, t, b_t)$
25: **end while**
26: **return** up to 16 stored proposals with the smallest objective values

Shading: principal changes from the campaign seed; routine definitions are in Appendix B.1.

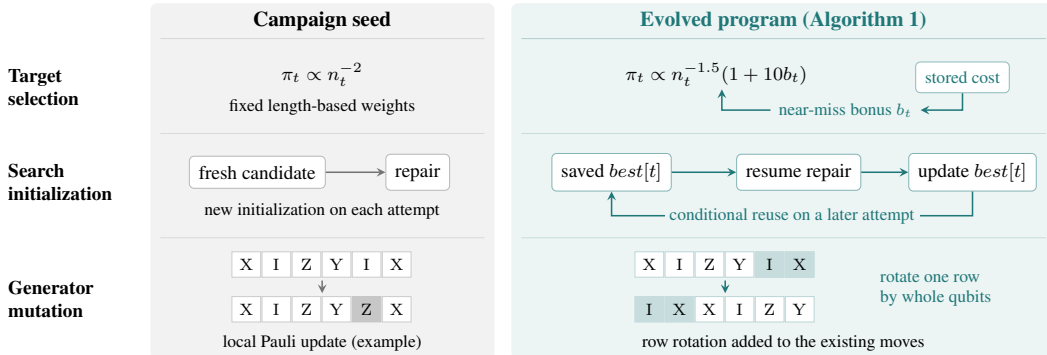


Figure 2: Program changes in Algorithm 1 (schematic).

4.1 DISTANCE IMPROVEMENTS

For listed cells, the tables give the July 2026 distance interval within the previous code parameters and our verified distance in Ours d (2026). Bold entries attain the applicable archived, EA-Plotkin or certified upper bound (Section 4.3). References identify previous lower-bound constructions and their dates. For $q = 4, 5$, where no comparable dated entry is archived, full constructed parameters appear in a separate *New constructions* block. No historical interval is assigned where no comparison entry exists.

Table 1: Qubit EAQECCs ($q = 2$).

Previous [[$n, k, d_{\ell}-d_u; c$]] $_q$	Ours d (2026)	References
[[7, 1, 5–6; 4]] $_2$	6	Grassl et al. (2020)
[[9, 1, 7–8; 6]] $_2$	8	Luo et al. (2022)
[[10, 1, 7–9; 6]] $_2$	9	Luo et al. (2022)
[[11, 1, 9–10; 8]] $_2$	10	Luo et al. (2022)
[[12, 1, 9–11; 8]] $_2$	11	Luo et al. (2022)
[[12, 2, 7–10; 9]] $_2$	9	Luo et al. (2022)
[[13, 1, 9–12; 8]] $_2$	11	Luo et al. (2022)
[[13, 1, 11–12; 10]] $_2$	12[†]	Luo et al. (2022)
[[15, 1, 13–14; 12]] $_2$	14[†]	Luo et al. (2022)

[†] Constructed by the evolved program at unseen code lengths.

Table 2: Qudit EAQECCs ($q = 3, 4, 5$).

Previous [[$n, k, d_{\ell}-d_u; c$]] $_q$	Ours d (2026)	References
[[9, 1, 7–8; 4]] $_3$	8	Luo et al. (2022)
[[10, 1, 8–9; 5]] $_3$	9	Luo et al. (2022)
[[11, 1, 9–10; 7]] $_3$	10	Luo et al. (2022)
[[13, 1, 10–12; 8]] $_3$	12	Luo et al. (2022)
<i>New constructions:</i> [[$n, k, d; c$]] $_q$		
[[10, 1, 9; 4]] $_4$	9	Ours
[[12, 1, 11; 6]] $_4$	11	Ours
[[14, 1, 13; 8]] $_4$	13	Ours
[[12, 1, 11; 5]] $_5$	11	Ours
[[14, 1, 13; 7]] $_5$	13	Ours

The tables report codes obtained across discovery campaigns, family constructions and solver refinement. Section 5 separately evaluates the programs generated in the controlled study. The daggered $n = 13, 15$ results show that the evolved program also constructs codes at lengths not used for training or selection (Section 5.2). The $n = 15$ row adds a finite construction to the table; these parameters are already covered by the family theorem and are not counted again among its 54 gap closures.

Our codes and the upper bounds close all nine listed qubit gaps in Table 1. At $(n, k, c) = (10, 1, 6)$ and $(12, 1, 8)$, the distance increases by two. Five odd-length family instances also close listed gaps and are included in the family count below. The two remaining qubit cells are settled by solver refinement (Section 4.3). Table 2 shows four qutrit gap closures, including $[[13, 1, 12; 8]]_3$, which raises the listed distance from 10 to 12 at fixed length and entanglement.

The previous $[[7, 1, 5; 4]]_2$ construction follows from the pure $[[11, 1, 5]]_2$ code of Calderbank et al. (1997) and the length-reduction rule of Grassl et al. (2020); we date this EAQECC construction by the 2020 rule. The other eight qubit baselines and the four displayed qutrit baselines follow from the 2022 version of Luo et al. (2022). Appendix A.4 records the source codes and propagation rules for each cell.

The 65 core qubit code records cover 61 distinct (n, k, c) cells: eight listed and 53 absent cells. The 27 qutrit codes comprise 15 improvements to listed entries, 11 constructions at absent cells and one code whose distance is already listed. Multiple codes at one cell are not counted as separate table entries. Four of these listed core cells overlap the family results in Section 4.2. The additional $n = 15$ code record is kept in the controlled-study archive, separately from these core-file counts.

4.2 CODE FAMILIES AND REDUCED ENTANGLEMENT

Theorem 1 gives $[[n, 1, n - 1; n - q - 1]]_q$ codes for $q \geq 3$ and $n \geq 2q + 1$, and for $q = 2$ at odd $n \geq 5$. The proof establishes the entire family; the auditor checks finite instances at $q = 2, 3, 4, 5$. The qubit instances close 29 listed gaps. At $q = 3$, increasing the supplied entanglement by one pair gives codes at $c = n - 3$, closing another 25 listed gaps. Together, these constructions settle 54 listed entries in the archived tables.

Additional constructions attain $d = n - 1$ at $c = n - q - 2$. The displayed $q = 4, 5$ examples include $[[10, 1, 9; 4]]_4$ and $[[12, 1, 11; 5]]_5$, requiring four and five shared pairs, respectively. These constructions establish achievable entanglement costs; smaller costs at $q = 4, 5$ have not been excluded. The archive contains 13 codes at $q = 4$ and 10 at $q = 5$. Table 2 selects those using the least entanglement reached at the displayed lengths.

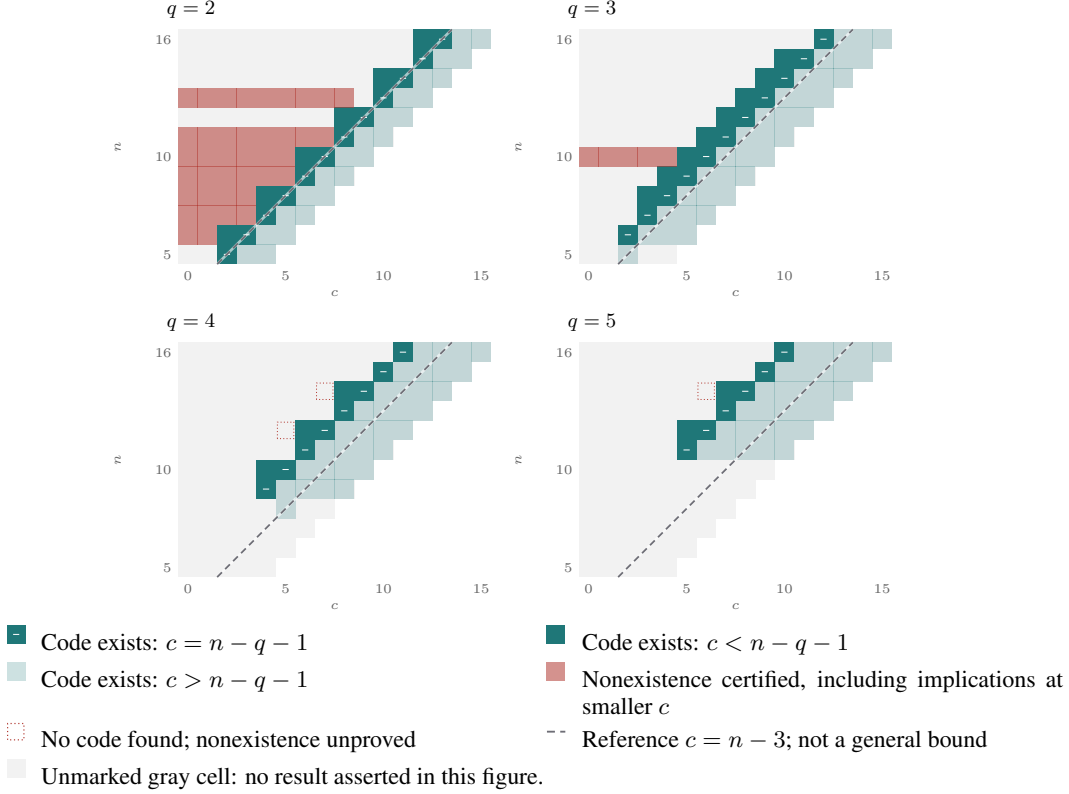


Figure 3: Linear stabilizer EAQECCs at $k = 1$, target distance $n - 1$.

Figure 3 concerns linear stabilizer EAQECCs with $k = 1$ and target distance $n - 1$. Teal at (q, n, c) denotes a construction with distance at least $n - 1$. The white tick, solid fill and pale fill distinguish $c = n - q - 1$, smaller c and larger c , respectively; these are positions relative to the family line, not claims of minimum entanglement. Pale cells correspond to explicit constructions or ebit lifting. Red cells are excluded by released, checked certificates or by their implications at smaller c through ebit lifting. A dotted outline records an unsuccessful search, not nonexistence. Unmarked gray cells assert neither construction nor exclusion. The dashed $c = n - 3$ line is the historical Brun–Devetak–Hsieh comparison, not a general bound in this regime (Appendix A.3).

4.3 UPPER BOUNDS AND OPTIMALITY

Solver refinement settles the two remaining qubit gaps from Table 1. Normal-form SAT finds $[[12, 2, 9; 9]]_2$, improving our previous $d = 8$ code. EA-Plotkin gives $d \leq \lfloor 144/15 \rfloor = 9$, so the new code is distance-optimal. At $(n, k, c) = (13, 1, 8)$, a DRAT-certified refutation excludes $d \geq 12$; our existing $d = 11$ code is therefore optimal. Both codes are checked by separate binary linear-algebra code, and DRAT-trim replays the nonexistence certificate. We attribute these two closures to solver refinement.

Matching nonexistence results also establish minimum entanglement at selected lengths. For qubits at $k = 1$, $d = n - 1$, the minimum is $n - 3$ at $n = 7, 9, 11$ and $n - 4$ at $n = 6, 8, 10$. For qutrits, the certified exclusion of $[[10, 1, 9; 4]]_3$ and a code at $c = 5$ give $c_{\min}^s(10, 1, 9) = 5$. These exclusions have checked DRAT certificates, with one large qubit proof regenerated on demand. Two further certified exclusions, $[[5, 2, 4; 3]]_2$ and $[[6, 2, 5; 4]]_2$, were also checked by exhaustive enumeration.

Applying the EA-Plotkin bound of Guo & Li (2013) additionally lowers 398 listed upper bounds. The auditor recomputes these corrections against the reference table data. These corrections follow from a known inequality and are reported as a separate source of upper bounds.

5 ABLATION STUDY

We test whether iterative feedback helps develop a researcher-proposed change into an effective search program. Both policies receive the same initial code and research guidance; their difference is how they reuse previous programs and execution feedback.

5.1 PROGRAM GENERATION

The shared proposal is to maintain and mutate a basis of $L = S^\perp$, converting it to S for exact evaluation. For $[[9, 1, 8; 6]]_2$ and $[[11, 1, 10; 8]]_2$, this replaces a basis of 14 or 18 generators with four vectors. Symplectic complementation preserves the feasible code set but changes the variables and local mutations. A preliminary calibration compares the initial S -side program with a working L -side control and fixes a budget at which the change can produce measurable success. Neither the control’s source nor generator matrices are supplied to the model.

Independent proposals always receive the initial program and its fixed training measurements. *Iterative evolution* receives the best-so-far program and the most recent attempt, including rejected attempts, with training feedback for both. The model can revise the representation, objective, mutation operators and restart policy. A proposal is a whole program revision; an evaluator call checks one candidate matrix produced by that program. Both use Gemini 3.5 Flash and identical tool access. There is no controller initialization: every success must be produced by the submitted program.

The balanced comparison contains four blocks and seven proposals per policy and block, 56 total. Unavailable model responses interrupted the planned 12-proposal runs; before held-out evaluation, a completion-only rule retained the common prefix. Training uses two seeds and 10,000 exact evaluations per target. Four new validation seeds select among each policy’s three training-best distinct programs; eight further seeds measure held-out performance. Only training feedback enters generation. Appendix D records calibration, selection and the interruption amendment.

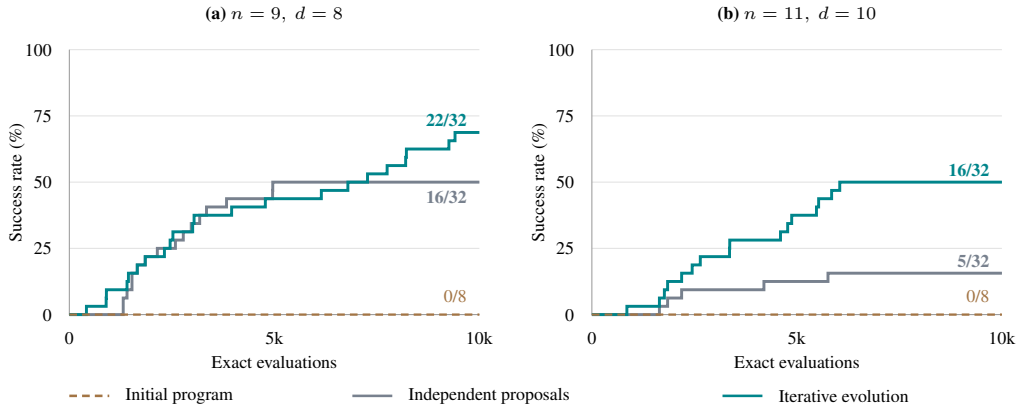


Figure 4: Search success under shared research guidance.

Figure 4 compares the initial program, independent proposals and iterative evolution. At 10,000 calls, success at $n = 9$ is 68.8% (22/32) for iterative evolution and 50.0% (16/32) for independent proposals; their curves cross at lower budgets. At $n = 11$, the corresponding rates are 50.0% (16/32) and 15.6% (5/32). Overall, iterative evolution succeeds in 38/64 executions (59.4%), versus 21/64 (32.8%) for independent proposals. The initial program records 0% (0/8) at each length on the same eight test seeds; each policy aggregates four selected programs, hence 32 executions per length. Mean normalized curve area increases from 0.239 to 0.358, with positive differences in two of four generation blocks. Charging unsuccessful executions the full allowance gives mean costs of 7,609.3

versus 6,421.5 evaluator calls. This measures verification-call efficiency; helper computation and wall time are reported separately.

5.2 GENERALIZATION TO LONGER CODES

The eight validation-selected programs are evaluated separately on $[[13, 1, 12; 10]]_2$ and $[[15, 1, 14; 12]]_2$, using four new seeds and 30,000 evaluations per target. These targets and their measurements are withheld during generation and selection. The later-generation program selected in block 3 succeeds in 75.0% (3/4) of executions at *each* length; its corresponding $n = 9, 11$ test result is 100% (16/16). The independent programs find no solutions at either longer length. Across all four selected programs per policy, success is 18.8% (6/32) versus 0% (0/32); all six successes come from that one iterative program.

Its preserved parent-child record shows changes beyond the initial representation conversion: stronger objective weights, a mixture of row replacement and row mixing, and backtracking with temperature adjustment from a saved local best (Algorithm 2). Successful construction takes 3,260–9,755 evaluations at $n = 13$ and 7,125–16,756 at $n = 15$. Each resulting code passes an independent linear-algebra check. Table 1 reports these code parameters alongside their prior distance intervals. Generalization of the search program is additional evidence beyond the family existence proof. The code record identifies the implemented changes; their individual causal effects are not separated by this policy comparison.

6 LIMITATIONS AND CONCLUSION

Limitations. The controlled study covers four generation blocks, one model and binary targets with known constructions. Its balanced comparison contains seven proposals per policy and block rather than the planned twelve because of unavailable model responses. Both policies receive the same explicit researcher proposal, so the contrast measures iterative development under that guidance. Mean improvement is not consistent across all blocks, and successes at longer lengths are concentrated in one selected program. Equal proposal and evaluator-call allowances also permit different token and CPU costs. Exact distance verification enumerates symplectic duals, so its cost grows exponentially with their dimension. The distance guarantees assume ideal decoding and noiseless receiver-side halves; circuit costs and hardware-level error rates are not evaluated. Our reproduction protocol checks outputs and fixed-program executions in recorded configurations; it does not recreate the original model sampling.

Conclusion. The pipeline supports a research process in which a researcher can propose a mathematical construction or representation, inspect verified outputs, and develop the idea through generated programs and exact execution feedback. The controlled study demonstrates this interface with sustained dual-space search: both policies implement the shared proposal, while iterative development yields higher mean held-out success and a preserved program that generalizes to two longer targets. The program sources, parent relationships and independently checked codes connect the research proposal to executable results.

The mathematical outputs include a human-derived family that, with ebit lifting, closes 54 listed gaps. Solver refinement establishes optimal distances 9 and 11 at $(n, k, c) = (12, 2, 9)$ and $(13, 1, 8)$; applying the known EA-Plotkin bound tightens 398 upper bounds. These results improve the documented tradeoff between error protection and communication resources. The generative component operates on search code, while the domain supplies the object representation, evaluator and certification rules. This interface makes the requirements for applying the pipeline to another discrete construction problem explicit. Archived programs, codes and proofs support further search and mathematical generalization.

AI USE STATEMENT

We used generative AI in the research workflow and in preparing this paper. The discovery campaigns used gemini-3.5-flash and gemini-3.1-pro-preview through AlphaEvolve; the controlled study used Gemini 3.5 Flash with a local controller. These models proposed and

revised executable search programs in response to researcher-supplied directions and exact evaluation feedback (Sections 3 and 5). LLM-based coding assistants, including Codex, also supported hypothesis and methodological refinement, experiment design and calibration, implementation of search, analysis, verification and certificate-generation tools, qualitative review of program changes, and interpretation of results. Assistance with mathematical claims and proof development included the coordinate-normalization argument in Lemma 2. Additional uses included data normalization and reformatting, literature search and synthesis, English drafting, translation and language editing of research discussions, manuscript restructuring and editing, and preparation of figures, tables and reproducibility artifacts. Generated codes are mathematical outputs whose parameters are independently checked. Verification uses exact finite-field computations, independent linear-algebra checks, Magma cross-checks and checked proof certificates as detailed in Appendices B–D; these computational checks do not mechanize every mathematical argument. Researcher guidance and mathematical generalization remain part of the disclosed workflow. The authors take responsibility for the final text, mathematical claims, proofs, code and reported results, including all AI-assisted contributions.

ETHICS STATEMENT

This work studies algebraic code constructions and search programs; it does not involve experiments on human participants or use participant-level or personal datasets. Its potential benefit is to improve methods for quantum error correction and communication, while the reported idealized parameters do not establish hardware-level reliability or operational security. Research-integrity considerations are addressed through dated sources, explicit provenance, independent verification, and separation of new parameter claims from retrospective reconstruction. Unsuccessful searches are distinguished from proved nonexistence. The controlled-study records preserve failed and excluded attempts, unavailable responses and protocol amendments, and its reviewer bundle omits account credentials and machine-local configuration. Researcher and AI contributions are disclosed, external sources are cited, and the artifact retains software licensing information. Compute allowances and model usage are documented in Appendices B.1 and D.

REPRODUCIBILITY STATEMENT

Appendix A specifies the code representation, mathematical assumptions, constructions and proofs, and the reference table data used for comparisons. Appendix B gives campaign configuration, preserved prompts, verification methods and refutation encodings, including the scope of certificate checks. Section 3.5 and Appendix C describe the auditor and its verification scope; the artifact README gives installation, integrity checks and fixed-program replay commands. Appendix D and the accompanying `experiments/hitl_ablation/` bundle preserve calibration and evaluation seeds, budgets, attempted requests, received program sources, parent relationships, selection records, interruption amendments and independently checked codes. The artifact supplies offline audit and replay entry points and is released under the MIT license. These records support re-computation of reported quantities and replay of preserved programs; fresh LLM sampling is not guaranteed to reproduce the same generated programs. External solver and Magma availability is reported separately from completed checks.

REFERENCES

- Todd Brun, Igor Devetak, and Min-Hsiu Hsieh. Correcting quantum errors with entanglement. *Science*, 314(5798):436–439, 2006. doi: 10.1126/science.1131563.
- A. R. Calderbank, E. M. Rains, P. W. Shor, and N. J. A. Sloane. Quantum error correction via codes over $GF(4)$. arXiv:quant-ph/9608006v5, 1997. URL <https://arxiv.org/abs/quant-ph/9608006v5>. Version dated 10 September 1997.
- Bogdan Georgiev, Javier Gómez-Serrano, Terence Tao, and Adam Zsolt Wagner. Mathematical exploration and discovery at scale. *arXiv preprint arXiv:2511.02864*, 2025.
- Markus Grassl. Entanglement-assisted quantum communication beating the quantum Singleton bound. *Physical Review A*, 103:L020601, 2021. doi: 10.1103/PhysRevA.103.L020601.
- Markus Grassl. Bounds on the parameters of various types of codes. (<http://www.codetables.de>), 2026. Accessed on 17 July 2026; table data preserved in the accompanying artifact.
- Markus Grassl, Felix Huber, and Andreas Winter. Entropic proofs of Singleton bounds for quantum error-correcting codes. arXiv:2010.07902v1, 2020. URL <https://arxiv.org/abs/2010.07902v1>. First public version, 15 October 2020; Theorem 7.
- Markus Grassl, Felix Huber, and Andreas Winter. Entropic proofs of Singleton bounds for quantum error-correcting codes. *IEEE Transactions on Information Theory*, 68(6):3942–3950, 2022. doi: 10.1109/TIT.2022.3149291.
- Luobin Guo and Ruihu Li. Linear Plotkin bound for entanglement-assisted quantum codes. *Physical Review A*, 87:032309, 2013. doi: 10.1103/PhysRevA.87.032309.
- Gaojun Luo, Martianus Frederic Ezerman, Markus Grassl, and San Ling. How much entanglement does a quantum code need? *arXiv preprint arXiv:2207.05647*, 2022. URL <https://arxiv.org/abs/2207.05647v1>. First public version, 12 July 2022; Table I and Section IV.
- Caroline Mauron, Terry Farrelly, and Thomas M. Stace. Optimization of tensor network codes with reinforcement learning. *arXiv preprint arXiv:2305.11470*, 2023.
- Alexander Novikov, Ngân Vū, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- Jan Olle, Remmy Zen, Matteo Puviani, and Florian Marquardt. Simultaneous discovery of quantum error correction codes and encoders with a noise-aware reinforcement learning agent. *npj Quantum Information*, 10:126, 2024.
- Hendrik Poulsen Nautrup, Nicolas Delfosse, Vedran Dunjko, Hans J. Briegel, and Nicolai Friis. Optimizing quantum error correction codes with reinforcement learning. *Quantum*, 3:215, 2019. doi: 10.22331/q-2019-12-16-215.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6.
- Jeonghwan Shin, Jun Heo, and Todd A. Brun. Entanglement-assisted codeword stabilized quantum codes. *Physical Review A*, 84:062321, 2011. doi: 10.1103/PhysRevA.84.062321.
- Vincent Paul Su, ChunJun Cao, Hong-Ye Hu, Yariv Yanay, Charles Tahan, and Brian Swingle. Discovery of optimal quantum error correcting codes via reinforcement learning. *Physical Review Applied*, 23:034048, 2025. doi: 10.1103/PhysRevApplied.23.034048.

Mark Webster and Dan Browne. Engineering quantum error correction codes using evolutionary algorithms. *arXiv preprint arXiv:2409.13017*, 2024. URL <https://arxiv.org/abs/2409.13017>.

Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *Theory and Applications of Satisfiability Testing (SAT)*, volume 8561 of *LNCS*, pp. 422–429, 2014.

Mark M. Wilde and Todd A. Brun. Optimal entanglement formulas for entanglement-assisted quantum coding. *Physical Review A*, 77:064302, 2008. doi: 10.1103/PhysRevA.77.064302.

A THE DOMAIN, THE TABLES, AND THE TARGETS

A.1 CODES AS SUBSPACES OF $\mathbb{F}_q^{2n}$

For prime-power q , our search uses $\mathbb{F}_q$ -linear stabilizer EAQECCs. Stabilizer generators are products of Pauli operators defining error-detection constraints. Entanglement assistance permits non-commuting generators on the transmitted qudits by completing them with operators on the receiver’s halves into commuting constraints. Their commutation relations determine the entanglement cost (Brun et al., 2006; Wilde & Brun, 2008). Nonadditive EAQECCs form a different search space (Shin et al., 2011).

We write vectors in bold, scalar coordinates and coefficients in italic, and subspaces with italic capitals such as S, L, R . Modulo phase, a Pauli operator is represented by $\mathbf{u} = (x_1, z_1, \dots, x_n, z_n) \in \mathbb{F}_q^{2n}$. For qubits, the pairs $(0, 0), (1, 0), (0, 1), (1, 1)$ represent I, X, Z, Y , respectively. The symplectic product with $\mathbf{v} = (x'_1, z'_1, \dots, x'_n, z'_n)$ is

$$\langle \mathbf{u}, \mathbf{v} \rangle = \sum_i (x_i z'_i - z_i x'_i),$$

and the symplectic weight $\text{wt}_s(\mathbf{u})$ counts positions with $(x_i, z_i) \neq (0, 0)$. For qubits, zero symplectic product means that the Pauli operators commute.

Let S be the generator span, with $s = \dim S$. Its symplectic dual $S^\perp$ consists of vectors orthogonal to S . The radical $S_{\text{iso}} = S \cap S^\perp$ contains stabilizer operations with trivial logical action. Vectors in $S^\perp \setminus S_{\text{iso}}$ represent undetectable errors with nontrivial logical action. The code parameters are

$$c = \frac{s - \dim S_{\text{iso}}}{2}, \quad k = n - s + c, \quad d = \min\{\text{wt}_s(\mathbf{v}) : \mathbf{v} \in S^\perp \setminus S_{\text{iso}}\}.$$

Finite-field linear algebra gives the dual and radical; enumerating the logical operators gives d exactly. The duals have at most 2^{20} elements at the sizes searched here. All parameters are thus computed from the generators, independently of the search program’s reports.

A.2 THE CLOSED FORM, AND WHY IT WORKS

Fix a prime power $q = p^e$, a length $n \geq 2q + 1$, and an enumeration $u_1, \dots, u_q$ of $\mathbb{F}_q$. On qudits $1, \dots, 2q$ take q disjoint pairs and let ρ_i be Z -type on pair i with z -values $(1, -1)$. Juxtaposition and powers below denote concatenation of coordinate pairs. Put

$$\mathbf{a} = (1, 0)^n, \quad \mathbf{b} = (u_1, 1)(u_1, z_2) \mid \cdots \mid (u_q, 1)(u_q, z_2) \mid (0, 1)^{n-2q-1}(0, \lambda),$$

with $z_2 = 1$ for odd p and $z_2 = 0$ for $p = 2$, and $L = \text{span}\{\mathbf{a}, \mathbf{b}, \rho_1, \dots, \rho_q\}$.

Theorem 1. For $q \geq 3$, and for $q = 2$ with n odd, $\lambda \in \mathbb{F}_q^\times$ can be chosen so that L is the normalizer side of an $[[n, 1, n-1; n-q-1]]_q$ EAQECC. For $q \geq 3$ these codes lie outside the region the BDH form describes as feasible, by exactly $q-2$ ebits; at $q = 2$ they saturate it.

Proof. The two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{F}_q^{2n}$ and the q radical candidates $\rho_1, \dots, \rho_q$ are defined above. Write $R_0 = \text{span}\{\rho_1, \dots, \rho_q\}$. Each ρ_i has two nonzero Z components on a distinct pair of qudits, so these q vectors are linearly independent. Moreover, $\mathbf{a}$ and $\mathbf{b}$ have equal X components within

each pair. Their symplectic products with ρ_i therefore vanish. The radical candidates are mutually orthogonal, since they are all Z -type.

The symplectic product of the two vectors is the sum of the Z components of $\mathbf{b}$:

$$\delta := \langle \mathbf{a}, \mathbf{b} \rangle = q(1 + z_2) + (n - 2q - 1)1_{\mathbb{F}_q} + \lambda = (n - 1)1_{\mathbb{F}_q} + \lambda.$$

Here integers are interpreted in $\mathbb{F}_q$, and $q1_{\mathbb{F}_q} = 0$ because its characteristic divides q . If $q \geq 3$, at most one of the $q - 1 \geq 2$ nonzero choices of λ makes $\delta = 0$; choose any other nonzero value. If $q = 2$ and n is odd, the only choice $\lambda = 1$ gives $\delta = 1$. Thus $\delta \neq 0$ in every case covered by the theorem.

To establish independence of the full generating set, suppose $\alpha\mathbf{a} + \beta\mathbf{b} + \sum_i t_i\rho_i = \mathbf{0}$. Since $n \geq 2q + 1$, the final qudit is outside the support of every ρ_i . Its two components are $(\alpha, \lambda\beta)$, so $\alpha = \beta = 0$. Independence of the radical candidates then implies $t_i = 0$ for every i . Consequently $\dim L = q + 2$. The restriction of the symplectic form to L , in this basis, has one nondegenerate 2×2 block $\begin{pmatrix} 0 & \delta \\ -\delta & 0 \end{pmatrix}$ and a $q \times q$ zero block. Hence its rank is two and $\text{rad}(L) = R_0$.

Every vector outside this radical has the unique form

$$\mathbf{w} = \alpha\mathbf{a} + \beta\mathbf{b} + \sum_{i=1}^q t_i\rho_i, \quad (\alpha, \beta) \neq (0, 0).$$

All $n - 2q$ tail qudits are nonzero: their components are (α, β) , except for the final $(\alpha, \lambda\beta)$. On pair i , the two coordinate pairs are

$$(\alpha + \beta u_i, \beta + t_i), \quad (\alpha + \beta u_i, \beta z_2 - t_i).$$

If $\beta = 0$, then $\alpha \neq 0$ and all n qudits are nonzero. If $\beta \neq 0$, exactly one index i satisfies $u_i = -\alpha/\beta$, since $u_1, \dots, u_q$ enumerate the field. Every other pair has two nonzero X components. On the exceptional pair, both Z components could vanish only if $t_i = -\beta = \beta z_2$. This is impossible because $1 + z_2 \neq 0$: it equals 2 in odd characteristic and 1 in characteristic two. At most one qudit can therefore be zero, giving $\text{wt}_s(\mathbf{w}) \geq n - 1$.

This lower bound is attained. Fix any index i , choose $\beta = 1$, $\alpha = -u_i$, $t_i = -1$, and $t_h = 0$ for $h \neq i$. The first qudit of pair i is zero and its second has nonzero Z component $1 + z_2$. All other pairs have nonzero X components, and the tail remains nonzero. This vector lies outside R_0 and has weight exactly $n - 1$.

Finally, set $S = L^\perp$. Nondegeneracy of the ambient symplectic form gives $S^\perp = L$ and $\dim S = 2n - q - 2$. Also $S \cap S^\perp = L^\perp \cap L = R_0$, whose dimension is q . The parameter formulas in Appendix A.1 now give

$$c = \frac{2n - q - 2 - q}{2} = n - q - 1, \quad k = n - (2n - q - 2) + c = 1, \quad d = n - 1.$$

For these parameters, $2(d - 1) - (n - k + c) = (2n - 4) - (2n - q - 2) = q - 2$. Thus the BDH expression is exceeded by $q - 2$ ebits for $q \geq 3$ and met with equality for $q = 2$, as stated. $\square$

The auditor instantiates this construction and verifies every member exactly for $q \in \{2, 3, 4, 5\}$ and $2q + 1 \leq n \leq 2q + 8$ (claim **families**); the theorem covers every prime power.

A.3 REFERENCE BOUNDS AND TARGET SELECTION

The reference tables record, for each parameter cell (q, n, k, c) , the listed lower and upper distance bounds and their construction sources. Our comparison uses the bounds accessed on 17 July 2026. The artifact preserves these table data and a second copy retrieved on 10 September 2026; both cover qubit codes with $n \leq 64$ and qutrit codes with $n \leq 36$. The reproduction command uses the July reference data unless another retrieval date is explicitly selected. Schema checks reject malformed records before classifying improvements or previously unlisted cells.

An entry with distance interval $[d_\ell, d_u]$ closes when construction and nonexistence bounds meet. For $(q, n, k, c) = (2, 7, 1, 4)$, a verified $[[7, 1, 6; 4]]_2$ code closes the listed $5 \leq d \leq 6$ gap. Cells absent from the reference tables are called *record targets* in the archive; absence alone does not establish novelty relative to all published constructions.

The familiar BDH Singleton expression $2(d - 1) \leq n - k + c$ is not a general bound in the large-distance regime (Grassl, 2021). Our principal family has $d = n - 1$, so we use the applicable generalized Singleton bounds of Grassl et al. (2022). The $[[12, 1, 11; 5]]_5$ code, for example, uses four fewer pairs than substitution into the BDH expression would suggest; it does not contradict a valid upper bound.

The target list is then everything still open under two filters. The EA-Plotkin bound (Guo & Li, 2013) gives $d \leq \lfloor 3 \cdot 4^{k-1} n / (4^k - 1) \rfloor$, which lowers the listed upper bound of 398 cells; EA-Singleton removes the rest. Cells we have separately proved empty are excluded as well. The effect is that a target is never impossible by a bound already known, so a search that returns nothing there is informative rather than expected.

A.4 SOURCES AND DATES OF PREVIOUS BOUNDS

The dates in Tables 1 and 2 refer to verified versions of public sources, not necessarily to the first discovery of each EAQECC parameter set. The reference qubit table fixes which lower and upper bounds we compare against. For $(7, 1, 4)$, the underlying pure $[[11, 1, 5]]_2$ code is recorded by Calderbank et al. (1997); applying Theorem 7 of Grassl et al. (2020) with $\ell = 4$ establishes the EAQECC lower bound by 15 October 2020. The elapsed time to 17 July 2026 is 5.8 years when rounded to one decimal.

The other eight qubit rows use the July 12, 2022 version of Luo et al. (2022). For odd $n \in \{9, 11, 13, 15\}$, Table I contains $[[n - 1, 0, n - 1; n - 3]]_2$. Quantum shortening gives $[[n - 2, 1, n - 2; n - 3]]_2$, and two length extensions yield the previous $[[n, 1, n - 2; n - 3]]_2$ bound. The two even-length rows arise similarly:

$$\begin{aligned} [[8, 0, 8; 6]]_2 &\longrightarrow [[7, 1, 7; 6]]_2 \longrightarrow [[10, 1, 7; 6]]_2, \\ [[10, 0, 10; 8]]_2 &\longrightarrow [[9, 1, 9; 8]]_2 \longrightarrow [[12, 1, 9; 8]]_2. \end{aligned}$$

A fourth length extension of $[[9, 1, 9; 8]]_2$ also gives the previous $[[13, 1, 9; 8]]_2$ bound. For the $k = 2$ row, the source’s $[[17, 0, 12; 9]]_2$ code gives

$$[[17, 0, 12; 9]]_2 \longrightarrow [[13, 4, 8; 9]]_2 \longrightarrow [[13, 2, 8; 9]]_2 \longrightarrow [[12, 2, 7; 9]]_2$$

by four quantum shortenings, taking a subcode, and puncturing. The rules and source codes are both present in the 2022 version, giving 4.0 years between publication and our table access date. These derivations match the provenance strings in the archived EAQECC table. They establish that the baselines were available by the stated dates and still listed in 2026; they do not establish a complete chronology of intervening work.

For the displayed qutrit rows, Table II of the same 2022 source contains $[[10, 0, 8; 4]]_3$, $[[10, 0, 9; 6]]_3$, and $[[13, 1, 10; 8]]_3$. Quantum shortening of the first gives $[[9, 1, 7; 4]]_3$; increasing its dimension with one additional shared pair gives $[[10, 1, 8; 5]]_3$. Applying the latter rule to $[[10, 0, 9; 6]]_3$ and then extending its length yields $[[11, 1, 9; 7]]_3$. The fourth baseline is the listed $[[13, 1, 10; 8]]_3$ code itself. The $q = 4, 5$ rows have no prior entry in the archived tables and are not included in these historical comparisons.

B THE PIPELINE IN DETAIL

B.1 CAMPAIGN CONFIGURATION AND VERIFICATION

Campaign configuration. Program mutations were proposed by two Gemini models (gemini-3.5-flash, gemini-3.1-pro-preview, sampled with equal weight) through Google Cloud Vertex AI’s AlphaEvolve service, with four generation and four evaluation workers running concurrently. The four campaigns were configured for 120, 120, 80 and 80 programs. Worker logs record 120, 121, 80 and 80 completed searches, respectively: 401 in total. The service counters are 121, 122, 81 and 81, each including an initial program entry with a supplied score. We use completed worker searches for budget accounting, giving 401×240 s, or 26.73 h of nominal search allowance. The experiment records, archived with the artifact, put model usage at 1.87 M input and 3.74 M output tokens in total. The first two campaigns shared one task specification and the last two another (Appendix B.3).

Discovery records and budgets. The retained campaign logs support nominal search allowances rather than elapsed discovery times. Campaign 1 starts 122 candidate searches and records 120 completions. Its first new $[[9, 1, 8; 6]]_2$ code immediately precedes completion 4, and the first $[[11, 1, 10; 8]]_2$ code completion 119. At 240s per completed search, these correspond to 0.267 h and 7.933 h of nominal allowance, excluding model generation, queueing, verification and in-flight work. The logs have no event timestamps and suppress later discoveries of the same parameter cell. A subsequent task specification names $[[7, 1, 6; 4]]_2$ as an earlier construction without preserving its first-hit budget. The $[[13, 1, 12; 10]]_2$ code record cites the closed-form theorem as provenance and is not counted as an original campaign discovery. The controlled reconstructions in Section 5 use newly generated, preserved programs and a separate protocol.

How Magma is made independent. The export carries generators only, so Magma re-derives every quantity we report. Symplectic weight is obtained by embedding $(x_i, z_i) \mapsto x_i + \omega z_i$ into $\text{GF}(q^2)$, so that Magma’s own `Weight()` does the counting: the check does not inherit our notion of weight, only our generators.

The archived construction program. Algorithm 1 summarizes the evolved construction program; its preserved source is linked from the artifact README. The generator matrix $\mathbf{G}$ spans S , and $\text{best}[t]$ stores the lowest-cost proposal seen for target t . The objective remains $5000|r.c - c_t| + 1000|r.k - k_t| + r.\text{offending}$, with cost 10^9 for an evaluator error. The campaign seed used fixed target weights n_t^{-2} . The evolved program uses normalized weights proportional to $n_t^{-1.5}(1 + 10b_t)$, initially with $b_t = 0$. After a repair attempt, `PROGRESSBONUS` uses the stored best cost o_t^* :

$$b_t \leftarrow \begin{cases} (1 + o_t^*)^{-1}, & 0 < o_t^* < 100, \\ 0, & o_t^* = 0, \\ b_t, & \text{otherwise.} \end{cases}$$

If no proposal was recorded, take $o_t^* = 10^9$. A proposal with zero cost at its initial evaluation takes the earlier **continue** branch and skips this bonus update, exactly as in the source.

For a new attempt, the program samples one scalar u and tests branches in order. If $u < 0.2$ and a stored positive-cost proposal exists, it resumes from that matrix. Otherwise, `CONSTRUCT` uses a cyclic construction when $u < 0.4$ and $s \leq 2n$, a divisor-stride construction when $u < 0.6$ and $s \leq 2n$, or a block construction when $u < 0.8$. If none applies, or the resulting rank is not s , it uses a random stabilizer. The construction branches and their validity checks remain subject to the same exact evaluator as the seed.

The cyclic branches use one or two nonzero base vectors. Row i takes base $i \bmod m$ and rotates it by $2h[i/m] \bmod 2n$ bits, where m is the number of bases. The ordinary cyclic branch sets $h = 1$; the added branch samples a positive proper divisor of n . The block branch extends the allowed dual dimension from 8 to 12 and block sizes from $\{2, 4, 6\}$ to $\{2, 4, 6, 8\}$.

During repair, `MIXEDMOVE` samples $v \in [0, 1)$. If $v < 0.2$ and $s \geq 2$, it XORs two distinct rows. Otherwise, if $v < 0.4$, it rotates one row by a uniformly chosen number of qubits from $1, \dots, n - 1$. The remaining branch applies a Pauli perturbation of weight 1–3 to one row. Thus the added operation moves a complete generator pattern; it is not a simultaneous coordinate permutation of the entire code. Rank-deficient proposals increment f and skip evaluation. The inherited annealing loop cools by a factor of 0.995 to a floor of 0.1, accepts by the Metropolis rule, and restarts after 1,500 attempts without an accepted strict improvement. The best accepted proposal for each target is retained; at most 16 are returned for external verification. Figure 2 is a schematic of these source-level changes, not a measurement of their individual performance effects.

B.2 THE REFUTATION ENCODING

The small side $L = S^\perp$ is encoded in symplectic normal form: $j = n - k - c$ radical generators $\rho_1, \dots, \rho_j$ together with k hyperbolic pairs (u_i, v_i) . Constant Gram constraints force the symplectic rank to be exactly $2k$ and the pair part to be independent. For qubits, the distance condition is that all $2^{2k+j} - 2^j$ non-radical combinations have symplectic weight at least d ; we express it with Gray-code chaining across combinations, so that consecutive combinations differ by one generator contribution, and `Sinz` cardinality counters for the weight itself.

For qubits, local symplectic transformations and coordinate permutations allow ρ_1 to be Z -only on a prefix. No corresponding coordinate restriction is imposed on the other radical generators.

For qutrits, each field element is one-hot encoded. The radical basis is in reduced row echelon form (RREF), and modular state chains impose its Gram constraints. For $k = 1$, it suffices to test the four projective logical representatives $u, v, u + v, u + 2v$, each shifted by every radical vector. Their nonzero scalar multiples have the same weights. At $d = n - 1$, pairwise clauses prevent a representative from vanishing at two coordinates. The following lemma permits an additional normalization.

Lemma 2 (Coordinate normalization). *Let $L \leq \mathbb{F}_q^{2n}$ have radical R with $\dim L = \dim R + 2$ and $\dim R > 2$. If every vector in $L \setminus R$ has symplectic weight at least $n - 1$, then local symplectic transformations can make R entirely Z -type.*

Proof. Write π_i for projection onto the two components at coordinate i , and π_{ih} for projection onto coordinates $i \neq h$. The distance condition implies $\ker(\pi_{ih}|_L) \subseteq R$, since a non-radical vector in this kernel would have weight at most $n - 2$. Thus the kernels on L and R coincide. Rank-nullity gives

$$\dim \pi_{ih}(L) - \dim \pi_{ih}(R) = \dim L - \dim R = 2.$$

The pair image lies in $\mathbb{F}_q^4$, so $\dim \pi_{ih}(R) \leq 2$. Suppose $\dim \pi_i(R) = 2$ for some i . For every $h \neq i$, the pair image then has dimension exactly two, and its projection onto $\pi_i(R)$ is an isomorphism. Consequently any vector in R vanishing at i vanishes at every h . This makes $\pi_i|_R$ injective, which would force $\dim R \leq 2$, a contradiction. Hence each $\pi_i(R)$ has dimension at most one. A determinant-one linear map of $\mathbb{F}_q^2$ sends each nonzero such line to the Z -axis; use the identity for a zero image. The product of these local maps preserves the symplectic form and every vector's support, and makes all radical X components zero. $\square$

For $[[10, 1, 9; 4]]_3$, the radical dimension is $j = 5$. After the local maps, row reduction preserves the zero X components, so the RREF encoding retains a representative of every possible code. We append $jn = 50$ unit clauses setting these components to zero. The resulting CNF has 33,033 variables and 476,271 clauses. CaDiCaL 3.0.0 produced a 30.3 MB binary DRAT proof in 14.2 s; DRAT-trim checked it independently in 84.8 s in the recorded run. The release includes the normalized CNF, compressed proof, checker log and transformation metadata. DRAT checks unsatisfiability of this CNF; the lemma establishes that its coordinate normalization loses no possible code. The auditor checks the unit clauses and hashes, rather than mechanically verifying the lemma. As a positive control, the normalized encoder remains SAT when fixed to the archived $[[10, 1, 9; 5]]_3$ code.

B.3 WHAT THE MODEL WAS TOLD

The specifications record two stages of researcher input. Campaigns 1 and 2 received human-proposed directions, including cyclic qubit shifts, while their initial program did not implement that construction. Campaigns 3 and 4 received the discovered instances, their human-derived generalization, and directions for further constructions. The task specification therefore supports HITL updates between campaigns. Website formatting is normalized in the excerpts below; all four specifications are archived verbatim. Later seeds and prompts contain the earlier findings and cannot serve as independent rediscovery baselines (Section 5).

Task specification, campaigns 1–2 — the runs that found the instances

```
Target: 68 open entries of the qubit EAQEC table (codetables.de).
Each target {n, k, c, d} asks for an entanglement-assisted quantum
code [[n,k,d;c]] whose existence is open [...] Required: symplectic
Gram rank exactly 2c on span S; then d = min symplectic weight over
S^perp \ S_iso must reach the target. [...] Promising directions:
constructive warm starts -- build S as c explicit hyperbolic pairs
plus n-k-c isotropic generators; derive candidates from classical
GF(4) additive codes (Wilde-Brun: c = rank of H H^dagger); modify
known-good stabilizer codes of nearby parameters; exploit that targets
sharing (n, c) can reuse structure; coupled multi-generator moves;
symmetry ansatz (cyclic qubit shifts). [...]
```

Task specification, campaigns 3–4 — after the finding was written back in

```
[...] Proven meta-lesson of this campaign: STRUCTURED CONSTRUCTION
PROGRAMS beat point search. A cyclic-shift ansatz (generators
= 2-bit cyclic shifts of one or two base Pauli strings) closed
[[7,1,6;4]], [[9,1,8;6]], [[11,1,10;8]] and was generalized by hand
into an infinite optimal family for odd  $n$ ; a block ansatz [...]
gave the even- $n$  branch. One good ansatz sweeps a whole ladder of
targets at once. Evolve NEW construction families: other shift
steps / quasi-cyclic orbits, multi-block radicals, GF(4)-additive
constructions, extension/shortening/puncturing of the already-closed
[[ $n,1,n-1;n-3$ ]] family [...]
```

C REPRODUCTION RESOURCES

The accompanying artifact’s `README.md` is the entry point for installation, result-specific verification commands and expected outputs. It links every table, figure and program result to its inputs and gives separate procedures for mathematical checks, certificate replay and fresh execution of the preserved programs.

The archive includes generator matrices, archived table data, proofs, model requests and responses, source code, seeds and selection records. Checksums identify the released inputs. The paper comparison is fixed by `artifacts/paper_reference.json`; using table data from a later retrieval date is an explicit choice. Section 3.5 describes fixed-program replay, and Appendix D specifies the controlled study.

Verification reports distinguish completed checks from missing tools. A solver verdict does not establish that a proof checker ran, and an archived Magma log is not a fresh cross-check. Overall **PARTIAL** means at least one requested check was not performed. These computational checks do not mechanize the mathematical proofs or reproduce the original LLM sampling.

D RESEARCH-GUIDED ABLATION DETAILS**D.1 SHARED RESEARCH GUIDANCE AND EVALUATION CALIBRATION**

The controlled study uses Gemini 3.5 Flash through Vertex `generateContent`, with a local program-development controller. It is distinct from the managed AlphaEvolve discovery campaigns. The two policies receive identical instructions to maintain and mutate $L = S^\perp$ throughout search and convert candidate bases to S using binary nullspace and symplectic-swap helpers. The initial code is the same S -side annealing callback in both policies. There is no protected initializer. Programs may change representations, objectives, mutation operators and restart schedules; the working L -side calibration code, archived codes and family formula are not provided to either policy.

Before model generation, four calibration seeds compare the initial callback with an interface-adapted, researcher-assisted L -side control at up to 10^5 evaluator calls. The controls depend on the call limit only through the remaining allowance, so their first-hit records give success curves at 1,000, 3,000, 10,000, 30,000 and 100,000 calls. At $n = 9$, the rule selects the smallest grid budget with initial success between 0.1 and 0.5, control success at least 0.5 and a difference of at least 0.25. A second cell may have zero initial success if the control succeeds in at least half the executions. The selected budget is 10,000: initial/control success is 1/4 versus 2/4 at $n = 9$, and 0/4 versus 2/4 at $n = 11$. Calibration is a test of whether the evaluator can reward a useful edit, not a measurement of evolutionary contribution. Independently verified codes establish feasibility at all training and transfer parameters before generation.

D.2 GENERATION, INTERRUPTION AND SELECTION

Four blocks each plan 12 proposals per policy. Independent proposals receive the fixed initial source and its fixed training feedback. Iterative evolution receives the best-so-far source and the latest attempt, including rejected attempts, with their training feedback. Training uses two fixed execution

seeds at $n = 9, 11$. Each proposal is ranked lexicographically by mean normalized success-curve area, success fraction and clipped distance ratio; earlier proposals break remaining ties. For first-hit call h under allowance B , the primary utility is

$$u(h; B) = \begin{cases} 1 - h/B, & \text{if the target is solved by call } B, \\ 0, & \text{otherwise.} \end{cases}$$

This equals the normalized integral of the success indicator over the available evaluator-call budget. The initial program remains eligible for selection. Proposal counts match; token and execution costs need not.

The first operational request was rejected for an out-of-range model seed, with no generated candidate. Its record is retained and the seed format was corrected. In the subsequent run, 58 model responses were received before two consecutive requests timed out. Neither ambiguous request was resent. An operational recovery attempt and its stopping rule are preserved. Before any validation or test observations, a completion-only amendment selects the longest contiguous prefix fully received and evaluated in every policy and block: seven proposals each, 56 total. The two additional completed proposals in one block are preserved but excluded symmetrically from this balanced analysis. The reported follow-up is therefore smaller than the 96-proposal plan. It follows earlier exploratory studies whose results motivated the calibration and shared guidance; those earlier measurements are not pooled into this comparison.

After generation ends, four new validation seeds at the training parameters select among each policy/block's three training-best distinct programs, using the same metric order and then training rank. The eight selected programs are distinct. Test uses eight further seeds at each training parameter and 10,000 calls per execution. The separate length transfer uses $n = 13, 15$, four seeds and 30,000 calls. Transfer measurements never enter selection. All seed lists, requests, source hashes, parent relationships, candidate outcomes and selections are retained in `experiments/hit1_ablation/`.

D.3 COMPLETE POLICY OUTCOMES

Table 3 gives the independent generation blocks. The primary paired mean difference is +0.1188, with two positive and two negative blocks. A descriptive two-sided sign-flip calculation over the four block differences gives 0.50; this small experiment does not establish statistically robust policy superiority. The execution seeds within a selected program are not independent generation replications. A sensitivity calculation excluding the block affected by the later unavailable responses gives a mean difference of +0.0819. All proposals included in the balanced comparison precede those unavailable responses.

Block	Successes /16		Mean curve area	
	Independent	Iterative	Independent	Iterative
1	4	3	0.180	0.066
2	4	10	0.177	0.407
3	5	16	0.202	0.596
4	8	9	0.398	0.362
Total/mean	21/64	38/64	0.239	0.358

Table 3: Held-out outcomes by generation block.

Figure 4 gives the full success curves. The initial program succeeds in 0/8 executions at each target. Counting a miss at the full allowance, mean charged evaluation cost is 7,609.3 for independent proposals and 6,421.5 for iterative evolution. Mean measured execution times are 1.86 s and 1.53 s, respectively, on the recorded machine; small timing differences are subject to machine and execution-order effects. Helper computations remain outside the evaluator-call count but inside the common resource limits. Each target/seed runs in a separate macOS sandbox with 120/125 s soft-/hard CPU limits, 135 s wall time and 1 GiB resident memory. Every explicit or returned-candidate verification consumes the same call quota.

D.4 PRESERVED PROGRAM AND LENGTH TRANSFER

The program selected in block 3 is `b02_evolution_g02`, whose preserved parent is `b02_evolution_g01`; generation indices start at zero. The parent already searches L and penalizes distance shortfall. The child increases the ebit-mismatch weight from 5,000 to 10,000 and the offending weight from 1 to 100. It also mixes row replacement, Pauli perturbations and basis-row mixing, scales mutation size with length, and restores a saved local best with a capped temperature adjustment after stagnation. Tinted expressions in Algorithm 2 mark these edits relative to this immediate parent; the inherited L -side representation and distance-shortfall term are not tinted. These changes are observed in the preserved source, rather than inferred from performance alone. This comparison does not isolate their individual causal effects.

Algorithm 2 Evolved search in the symplectic dual.

Require: target (n, k_t, c_t, d_t) , exact-evaluation budget N

$$\Phi(R) = 10^4 |R.c - c_t| + 100 R.\text{offending} + 10^3 \Delta_d(R)$$

▷ Increase penalties for entanglement mismatch and low-weight violations.

```

1:  $r \leftarrow n + k_t - c_t$ 
2: while budget remains do
3:    $L \leftarrow$  a random independent list of  $r$  Pauli vectors
4:    $C \leftarrow \Phi(\text{EVAL}(L^\perp)); (L_*, C_*) \leftarrow (L, C)$ 
   ▷ Retain the best state within each restart.
5:    $\tau \leftarrow 50; f \leftarrow 0; g \leftarrow 0$ 
6:   while  $f < 600$  and budget remains do
7:      $\tau \leftarrow \max(0.01, 0.995\tau)$ 
8:      $L' \leftarrow \text{MIXEDMOVE}(L)$ 
   ▷ Add row mixing and allow larger Pauli changes at longer lengths.
9:     if  $\text{rank } L' \neq r$  then  $f \leftarrow f + 1$ ; continue
10:     $C' \leftarrow \Phi(\text{EVAL}((L')^\perp)); \delta \leftarrow C' - C$ 
11:    if  $\delta \leq 0$  or  $U(0, 1) < \exp(-\delta/\tau)$  then
12:       $(L, C) \leftarrow (L', C')$ 
13:      if  $C < C_*$  then
14:         $(L_*, C_*) \leftarrow (L, C); g \leftarrow 0$ 
15:      else
16:         $g \leftarrow g + 1$ 
17:      end if
18:       $f \leftarrow 0$  if  $\delta < 0$ , otherwise  $f \leftarrow f + 1$ 
19:    else
20:       $f \leftarrow f + 1; g \leftarrow g + 1$ 
21:    end if
22:    if  $g > 80$  then
   ▷ On stagnation, return to the best state and adjust the temperature.
23:       $(L, C) \leftarrow (L_*, C_*); g \leftarrow 0$ 
24:       $\tau \leftarrow \min(30, 1.5\tau)$ 
25:    end if
26:  end while
27: end while
28: return the evaluator's best verified record, if any

```

Shading: changes from the immediate parent; exact definitions in Appendix D.4.

The program cost in Algorithm 2 is

$$\Phi(R) = 10^4 |R.c - c_t| + 100 R.\text{offending} + 10^3 \Delta_d(R),$$

where $\Delta_d(R) = \max(0, d_t - R.d)$, or d_t if distance is unavailable. If the offending count or entanglement cost is missing, the score is 10^9 . These definitions match the preserved source.

In Algorithm 2, a random Pauli perturbation has weight uniformly sampled from $1, \dots, \max(2, \lfloor n/2 \rfloor) - 1$. MIXEDMOVE chooses a row uniformly: with probability 0.3 it replaces the row by such a perturbation, with probability 0.3 it XORs the perturbation into the row, and otherwise it samples a second row. If that row is different, it XORs the two rows and, with probability 0.5, adds a Pauli perturbation. A rank-deficient proposal increments f and skips

evaluation. Otherwise, f resets only on an accepted improvement over the current cost, and increments on all other moves. The counter g resets on a new restart-local best and increments on other evaluated moves. At $g > 80$, the program restores that best, sets $g = 0$ and applies $\tau \leftarrow \min(30, 1.5\tau)$. This adjustment can lower a temperature still above 30; it is not always a temperature increase. The source and its budget behavior remain unchanged.

The child succeeds in 100% (16/16) of primary-test executions and 75.0% (3/4) at each longer-length target. Its successful $n = 13$ executions take 3,260, 5,263 and 9,755 calls; at $n = 15$ they take 7,125, 15,789 and 16,756. The two remaining executions exhaust 30,000 calls. The other three iterative selections and all four independent selections record zero transfer successes, giving policy totals of 6/32 and 0/32. All eight selected programs implement sustained L -side repair; none computes a full private distance test and invokes the common evaluator only on an already filtered solution. Two iterative selections are first proposals, so their implementations do not require feedback from earlier generated candidates.

D.5 OFFLINE VERIFICATION

The offline audit reconstructs all 56 included proposal inputs and parent relationships, checks the validation selections, and independently verifies 235 saved code records. These files include repeated outcomes across calibration, training, validation and test runs, rather than 235 distinct parameter sets. Failed and excluded attempts and both unavailable model responses remain in the artifact. The recorded environment uses Python 3.12.12 and NumPy 2.5.1; the artifact README provides the audit and fixed-program replay commands. Model program bytes, numerical observations and generator matrices are preserved, while machine-local paths and opaque provider identifiers are omitted from the reviewer bundle.